Learning Objectives

Learners will be able to...

- **Define merging a graph**
- **Merge overlapping vertices**
- **Rename overlapping vertices**
- **Handle data for overlapping vertices**

Graph Merging Techniques and Challenges

Graph Merging Techniques and Challenges

Graph merging is a fundamental operation in graph theory, often required in scenarios where separate sets of data need to be combined into a unified structure. This page will discuss how to merge two graphs, focusing on a method that involves pointing nodes from one graph to another. We'll also address challenges like overlapping vertices.

Merging Graphs by Pointing Nodes

The idea of merging two graphs involves creating connections from the nodes of one graph to the nodes of another. This method is particularly straightforward in directed graphs, where edges inherently have a direction, but it can also be applied to undirected graphs.

Implementation Approach

1. **Identify Corresponding Vertices**: Determine which vertices in the two graphs correspond to each other, if any. This might involve matching vertices based on some shared property or identifier.
2. **Create New Edges**: For each pair of corresponding vertices, create an edge from a vertex in one graph to the corresponding vertex in the other. This effectively "points" vertices from one graph to another, merging them.
3. **Handle Non-Corresponding Vertices**: For vertices that don't have a counterpart in the other graph, decide whether to include them independently or establish some form of default connection.

Example

Consider two graphs, G1 and G2, where some vertices in G1 correspond to vertices in G2:



Merging G1 into G2 might result in connections like A → X, B → Y, and C → Z.

Challenges in Graph Merging

Overlapping Vertices

One of the main challenges in merging graphs is handling overlapping vertices, i.e., vertices that are common to both graphs. This situation can lead to ambiguity about which connections should be retained or how the properties of these overlapping vertices should be merged. Strategies to address this challenge include:

1. **Vertex Merging**: Combine the overlapping vertices into a single vertex. This approach may involve aggregating or reconciling the properties and edges of the combined vertices.
2. **Vertex Duplication**: Keep overlapping vertices separate but identify them as related. This can preserve the original structure of each graph but may lead to redundancy.
3. **Conflict Resolution Logic**: Implement a set of rules or logic to resolve conflicts arising from overlapping vertices, such as prioritizing one graph's structure over the other or creating a new vertex that combines aspects of both.

Consistency in Edge Direction

For directed graphs, ensuring consistency in the direction of edges post-merging is important, especially when the original graphs have different orientations or structures. Care must be taken to preserve the semantic meaning of edge directions.

Handling Different Graph Types

Merging graphs of different types (e.g., a weighted graph with an unweighted graph) poses another challenge. One must decide how to handle differing edge properties, such as whether to assign default weights to unweighted edges.

Practical Considerations

- **Use Case Analysis**: Before merging, analyze the use case to determine the most appropriate merging strategy. This analysis should consider how the merged graph will be used and what information it should convey.
- **Data Integrity**: Ensure that the merging process does not inadvertently distort the data represented by the original graphs. Maintaining data integrity is crucial for the merged graph to be useful and reliable.

- **Algorithmic Complexity**: Consider the computational complexity of the merging process, especially for large graphs. Efficient algorithms and data structures are key to handling large-scale merges effectively.

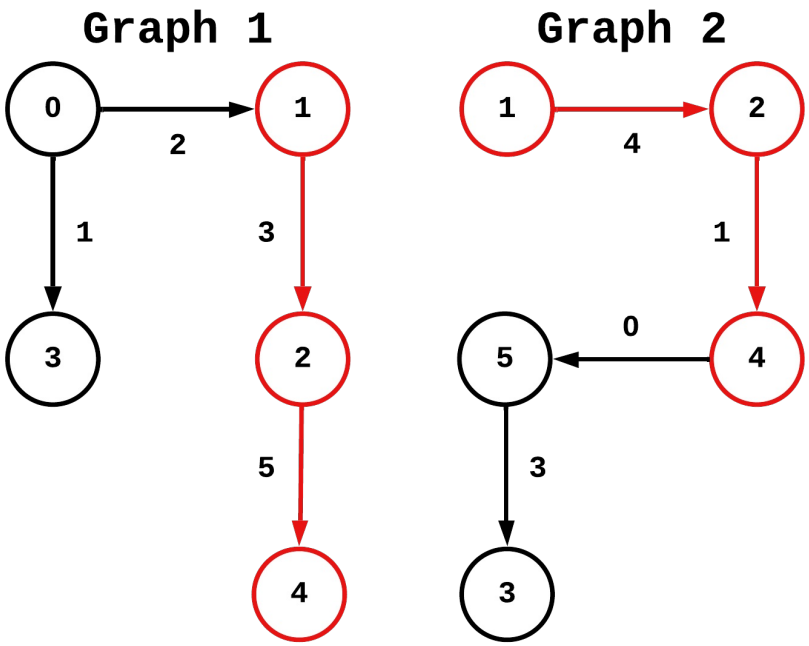
Graph merging is a powerful technique in data structure manipulation, useful in various scenarios like network construction, data integration, and problem-solving in graph theory. Understanding how to effectively merge graphs while addressing challenges like overlapping vertices and maintaining data integrity is crucial for anyone working with complex graph-based systems.

Merging Vertices

Group Overlapping Vertices

When merging two `graphs`, a common challenge is dealing with overlapping vertices, that is vertices that are present in both graphs. Handling these requires careful consideration to ensure the merged graph accurately represents the combined structure. The next few pages will explore strategies to handle overlapping `vertices`.

One way to handle overlapping vertices is to merge them as a single set of neighbors. Consider the following graphs (overlapping vertices are in red):



We see overlapping vertices in the edges `1 -> 2` and `2 -> 4`. In this strategy, we are going to place these overlapping vertices in the same location. That is, vertex `2` will have the neighbor `4` with a weight of `1` and another neighbor `4` with a weight of `5`. Similarly, vertex `1` will have the neighbor `2` with a weight of `3` and another neighbor `2` with a weight of `4`.

▼ **Understanding the Graph**

> The `Graph` object is composed of a list of `Vertex` objects. A Vertex has an `id` which is the integer identifier for the vertex. It also has a list of neighbors (Vertex objects) and a list of `Edge` objects. Each Edge object has a Vertex object representing the end point of the edge and an integer representing the weight of the edge. The code for these classes is found toward the bottom of the code editor. Understanding the implementation of the `Graph`, `Vertex`, and `Edge` classes will help you merge two graphs together. The `main` method that tests our code has already been completed.

Start by creating the `GraphMerger` class. It has two static methods, `addEdge` and `mergeGraphs`. The `addEdge` method takes a Graph object, an integer representing an index, a `Vertex` object representing the current vertex, another `Vertex` object representing the neighbor, and a vector of Vertex objects representing all of the neighbors of the current vertex. This method does not return a value. The `mergeGraphs` method takes two `Graph` objects and returns a `Graph` object.

```
class GraphMerger {
public:
    static void addEdge(Graph& graph,
                        int sourceIndex,
                        const Vertex& currentVertex,
                        const Vertex* neighbor,
                        const std::vector<Vertex*>& neighbors) {


    }

    static Graph mergeGraphs(const Graph& graph1, const Graph&
        graph2) {


    }
};
```

The `addEdge` method creates a weighted edge in the merged graph between vertices. We are going to use indices to help us locate the correct Vertex object. The source index (first vertex in an edge) has already been passed to the method. Calculate the index of the target (second vertex in an edge) for the neighbor. Next, get the weight for the neighbor. Finally, add an edge between the source vertex and the target vertex.

```
    static void addEdge(Graph& graph,
                        int sourceIndex,
                        const Vertex& currentVertex,
                        const Vertex* neighbor,
                        const std::vector<Vertex*>& neighbors) {
        int targetIndex = neighbor->getId();
        auto it = std::find(neighbors.begin(), neighbors.end(),
        neighbor);
        if (it != neighbors.end()) {
            int neighborIndex = std::distance(neighbors.begin(),
        it);
            int weight = currentVertex.getWeight(neighborIndex);
            graph.addEdge(sourceIndex, targetIndex, weight);
        }
    }
```

The `mergeGraphs` method takes two `Graph` objects and returns a merged Graph object. Start by calculating the largest size between the two graphs to be merged. Then create the `mergedGraph` variable which is a Graph object that uses the largest size from the two given graphs. The actual merging is a two-step process. Start by iterating over the first graph. Get each vertex in the graph. Then generate a list of neighbors for each vertex. Iterate over the neighbors, adding an edge between the current vertex and each neighbor.

```
    static Graph mergeGraphs(const Graph& graph1, const Graph&
        graph2) {
        int totalVertices = std::max(graph1.getSize(),
        graph2.getSize());
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); ++i) {
            const Vertex& vertex = graph1.getVertices().at(i);
            const std::vector<Vertex*>& neighbors =
        vertex.getNeighbors();
            for (const Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
            }
        }

        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); ++i) {
            const Vertex& vertex = graph2.getVertices().at(i);
            const std::vector<Vertex*>& neighbors =
        vertex.getNeighbors();
            for (const Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
            }
        }

        return mergedGraph;
    }
```

▼ Code

Your code should look like this:

```
class GraphMerger {
public:
    static void addEdge(Graph& graph,
                        int sourceIndex,
                        const Vertex& currentVertex,
                        const Vertex* neighbor,
                        const std::vector<Vertex*>&
        neighbors) {
        int targetIndex = neighbor->getId();
        auto it = std::find(neighbors.begin(),
        neighbors.end(), neighbor);
        if (it != neighbors.end()) {
            int neighborIndex =
        std::distance(neighbors.begin(), it);
            int weight =
        currentVertex.getWeight(neighborIndex);
            graph.addEdge(sourceIndex, targetIndex,
        weight);
        }
    }

    static Graph mergeGraphs(const Graph& graph1, const
        Graph& graph2) {
        int totalVertices = std::max(graph1.getSize(),
        graph2.getSize());
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); ++i) {
            const Vertex& vertex =
        graph1.getVertices().at(i);
            const std::vector<Vertex*>& neighbors =
        vertex.getNeighbors();
            for (const Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
            }
        }

        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); ++i) {
            const Vertex& vertex =
        graph2.getVertices().at(i);
            const std::vector<Vertex*>& neighbors =
        vertex.getNeighbors();
            for (const Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
            }
        }

        return mergedGraph;
    }
};
```

Run the program. Remember, the two graphs share edges between vertices 1 and 2 as well as between 2 and 4. This solution should show two different edges between vertices 1 and 2 as well as two edges between vertices 2 and 4. The weight of each edge is listed between parentheses.

You should see the following output:

```
Graph 1:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3)
Vertex 2: -> 4 (5)
Vertex 3:
Vertex 4:
Vertex 5:

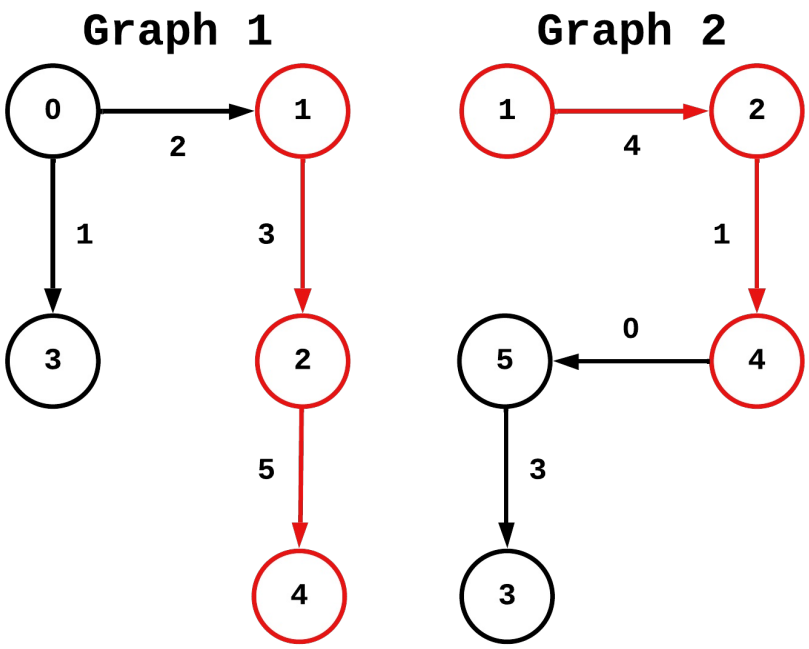
Graph 2:
Vertex 0:
Vertex 1: -> 2 (4)
Vertex 2: -> 4 (1)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)

Merged Graph:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3) -> 2 (4)
Vertex 2: -> 4 (5) -> 4 (1)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)
```

Renaming Vertices

Rename All Vertices

One way to handle overlapping `vertices` is to rename all of the `vertices` and `edges` from the second `graph`. Consider the following graphs (overlapping vertices are in red):



In this strategy, the `vertices` in the second graph will be given a different `value`. The change in value must be consistent when renaming the vertices. The renamed vertices are then added to a larger graph. In effect, the merged graph will contain two parallel graphs. The first graph will keep all of its original values, while the second graph will have a constant added to all of its values.

Begin by creating the `GraphMerger` class that has two methods, `addEdge` and `mergeGraphs`. The `addEdge` method is very similar to the version from the previous page. This version has the extra parameter offset, an integer value used to rename vertices from the second graph. The method signature for `mergeGraphs` is the same as the previous page.

```
class GraphMerger {

public:
    static void addEdge(Graph& graph,
        int index,
        Vertex& currentVertex,
        Vertex* neighbor,
        std::vector<Vertex*>& neighbors,
        int offset
    ) {



    }

    static Graph mergeGraphs(Graph& graph1, Graph& graph2) {



    }
};
```

For the `addEdge` method, it will begin much like the method from the previous page. However, when calculating `targetIndex`, you are going to get the `ID` from the `neighbor` and add `offset` to it. Finish the method by getting the `index` of the `neighbor`, the `weight` of the `edge`, and adding a new `edge` to the `merged` graph.

```
    static void addEdge(Graph& graph,
        int sourceIndex,
        Vertex& currentVertex,
        Vertex* neighbor,
        std::vector<Vertex*>& neighbors,
        int offset
    ) {
        int targetIndex = neighbor->getId() + offset;
        auto it = std::find(neighbors.begin(), neighbors.end(),
        neighbor);
        int neighborIndex = std::distance(neighbors.begin(),
        it);
        int weight = currentVertex.getWeight(neighborIndex);
        graph.addEdge(sourceIndex, targetIndex, weight);
    }
```

Inside the `mergeGraphs` method, we are going to calculate the `sum` of `vertices` for the `first` graph and `second` graph. Then create a `new` Graph object with this `size`. The merged graph needs to be large enough to contain both the `first` and `second` graphs without any overlap. Just as before, iterate over `graph1` and add each edge to the merged graph. Remember, `addEdge` has an extra parameter. Be sure to pass `0` to the method so that the original values are not changed.

```
    static Graph mergeGraphs(Graph& graph1, Graph& graph2) {
        int totalVertices = graph1.getSize() + graph2.getSize();
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); i++) {
            Vertex& vertex = graph1.getVertices().at(i);
            std::vector<Vertex*> neighbors =
        vertex.getNeighbors();
            for (auto& neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors, 0);
            }
        }


    }
```

Then iterate over `graph2`. We are going to add each `edge` from this graph to the `merged` graph. However, we need to calculate an offset value so we can rename all of the `vertices` in the `second` graph. When calling `addEdge`, be sure to pass it the `size` of the `first` graph. This value will be used to maintain a `second`, parallel `graph`.

```
        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); i++) {
            Vertex& vertex = graph2.getVertices().at(i);
            int vertexIndex = graph1.getSize() + i;
            std::vector<Vertex*> neighbors =
        vertex.getNeighbors();
            for (auto& neighbor : neighbors) {
                addEdge(mergedGraph, vertexIndex, vertex,
        neighbor, neighbors, graph1.getSize());
            }
        }

        return mergedGraph;
    }
```

▼ Code

> Your code should look like this:

```
class GraphMerger {

public:
    static void addEdge(Graph& graph,
        int sourceIndex,
        Vertex& currentVertex,
        Vertex* neighbor,
        std::vector<Vertex*>& neighbors,
        int offset
    ) {
        int targetIndex = neighbor->getId() + offset;
        auto it = std::find(neighbors.begin(),
        neighbors.end(), neighbor);
        int neighborIndex =
        std::distance(neighbors.begin(), it);
        int weight =
        currentVertex.getWeight(neighborIndex);
        graph.addEdge(sourceIndex, targetIndex, weight);
    }


    static Graph mergeGraphs(Graph& graph1, Graph& graph2)
        {
        int totalVertices = graph1.getSize() +
        graph2.getSize();
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); i++) {
            Vertex& vertex = graph1.getVertices().at(i);
            std::vector<Vertex*> neighbors =
        vertex.getNeighbors();
            for (auto& neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors, 0);
            }
        }

        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); i++) {
            Vertex& vertex = graph2.getVertices().at(i);
            int vertexIndex = graph1.getSize() + i;
            std::vector<Vertex*> neighbors =
        vertex.getNeighbors();
            for (auto& neighbor : neighbors) {
                addEdge(mergedGraph, vertexIndex, vertex,
        neighbor, neighbors, graph1.getSize());
            }
        }

        return mergedGraph;
    }
};
```

Run the program. Because the `first` graph has a size of `6`, the `vertices` for the second graph will be their original value plus `6` (vertex 1 becomes vertex 7, vertex 2 becomes vertex 8, etc). The `weights`, however, should not change.

You should see the following output:

```
Graph 1:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3)
Vertex 2: -> 4 (5)
Vertex 3:
Vertex 4:
Vertex 5:

Graph 2:
Vertex 0:
Vertex 1: -> 2 (4)
Vertex 2: -> 4 (1)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)

Merged Graph:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3)
Vertex 2: -> 4 (5)
Vertex 3:
Vertex 4:
Vertex 5:
Vertex 6:
Vertex 7: -> 8 (4)
Vertex 8: -> 10 (1)
Vertex 9:
Vertex 10: -> 11 (0)
Vertex 11: -> 9 (3)
```

Rename Overlapping Vertices

Instead of renaming all of the vertices from the second graph, we can modify our code to only rename those vertices which are overlapping. Find the second for loop in the mergeGraphs method (this is the loop that iterates over graph2). In the inner for loop, add an if-else statement. If the current vertex is not overlapping, then add the edge with an offset of 0. Otherwise, add the edge with an offset of the size of the first graph. We know if a vertex is not overlapping when there are no neighbors for a given vertex in the merged graph (mergedNeighbors.empty()) but there are neighbors in the second graph (!neighbors.empty()). Finally, return the merged graph.

```
        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); i++) {
            Vertex& vertex = graph2.getVertices()[i];
            int vertexIndex = graph1.getSize() + i;
            std::vector<Vertex*> neighbors =
vertex.getNeighbors();
            std::vector<Vertex*> mergedNeighbors =
mergedGraph.getVertices()[i].getNeighbors();
            for (Vertex* neighbor : neighbors) {
                if (mergedNeighbors.empty() &&
!neighbors.empty()) {
                    addEdge(mergedGraph, i, vertex, neighbor,
neighbors, 0);
                } else {
                    addEdge(mergedGraph, vertexIndex, vertex,
neighbor, neighbors, graph1.getSize());
                }
            }
        }

        return mergedGraph;
```

▼ Code

> Your code should look like this:

```
class GraphMerger {

public:
    static void addEdge(Graph& graph,
        int sourceIndex,
        Vertex& currentVertex,
        Vertex* neighbor,
        std::vector<Vertex*>& neighbors,
        int offset
    ) {
        int targetIndex = neighbor->getId() + offset;
        auto it = std::find(neighbors.begin(),
        neighbors.end(), neighbor);
        int neighborIndex =
        std::distance(neighbors.begin(), it);
        int weight =
        currentVertex.getWeight(neighborIndex);
        graph.addEdge(sourceIndex, targetIndex, weight);
    }

    static Graph mergeGraphs(Graph& graph1, Graph& graph2)
        {
        int totalVertices = graph1.getSize() +
        graph2.getSize();
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); i++) {
            Vertex& vertex = graph1.getVertices()[i];
```

```
            std::vector<Vertex*> neighbors =
        vertex.getNeighbors();
            for (Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors, 0);
            }
        }

        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); i++) {
            Vertex& vertex = graph2.getVertices()[i];
            int vertexIndex = graph1.getSize() + i;
            std::vector<Vertex*> neighbors =
        vertex.getNeighbors();
            std::vector<Vertex*> mergedNeighbors =
        mergedGraph.getVertices()[i].getNeighbors();
            for (Vertex* neighbor : neighbors) {
                if (mergedNeighbors.empty() &&
        !neighbors.empty()) {
                    addEdge(mergedGraph, i, vertex,
        neighbor, neighbors, 0);
                } else {
                    addEdge(mergedGraph, vertexIndex,
        vertex, neighbor, neighbors, graph1.getSize());
                }
            }
        }

        return mergedGraph;
    }
};
```

Run the program. Since edges $4 \to 5$ and $5 \to 3$ are unique to the second graph, the vertices should not be renamed. However, edges $1 \to 2$ and $2 \to 4$ appear in both graphs. So the vertices 1 and 2 will be renamed. The weights should not change.

You should see the following output:

```
Graph 1:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3)
Vertex 2: -> 4 (5)
Vertex 3:
Vertex 4:
Vertex 5:

Graph 2:
Vertex 0:
Vertex 1: -> 2 (4)
Vertex 2: -> 4 (1)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)

Merged Graph:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3)
Vertex 2: -> 4 (5)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)
Vertex 6:
Vertex 7: -> 8 (4)
Vertex 8: -> 10 (1)
Vertex 9:
Vertex 10:
Vertex 11:
```

Handling Data

Summing the Weights

The final strategy we are going to discuss is handling data vertices. There is no hard and fast rule for how to handle the data. This depends on the needs of your project. For this example, we are going to combine the weights for any overlapping vertices. What is important, however, is that you may find yourself needing to write custom logic to handle overlapping vertices when merging two graphs.

In this strategy, there will be no duplicate or renamed edges in the merged graph. All edges from both graphs will be present, but the weights will change.

Begin by creating the `GraphMerger` class. This time, however, the class has three methods–`addEdge`, `containsEdge`, and `mergeGraphs`. The `addEdge` and `mergeGraphs` operate in a manner similar to what we have seen before. The `containsEdge` takes a `Vertex` object and a list of `Vertex` objects and returns a boolean.

```
class GraphMerger {

public:
    static void addEdge(Graph& graph,
        int index,
        Vertex* currentVertex,
        Vertex* neighbor,
        const std::vector<Vertex*>& neighbors
    ) {



    }

    static bool containsEdge(Vertex* v, const
        std::vector<Vertex*>& mergedEdges) {



    }

    static Graph mergeGraphs(const Graph& graph1, const Graph&
        graph2) {


    }
};
```

Just as before, the `addEdge` method adds a new edge between vertices to the merged graph. Note, the `addEdge` method does not take an offset value for this implementation.

```
    static void addEdge(Graph& graph,
        int index,
        Vertex* currentVertex,
        Vertex* neighbor,
        const std::vector<Vertex*>& neighbors
    ) {
        int sourceIndex = index;
        int targetIndex = neighbor->getId();
        auto it = std::find(neighbors.begin(), neighbors.end(),
        neighbor);
        int neighborIndex = std::distance(neighbors.begin(),
        it);
        int weight = currentVertex->getWeight(neighborIndex);
        graph.addEdge(sourceIndex, targetIndex, weight);
    }
```

The `containsEdge` method determines if a given edge is present in a set of neighboring vertices. Iterate over the list of `Vertex` pointers. Check to see if the ID of the given vertex is equal to the current edge. If so, return `true`. After the loop ends, return `false`.

```
    static bool containsEdge(Vertex* v, const
        std::vector<Vertex*>& mergedEdges) {
        for (const auto& edge : mergedEdges) {
            if (edge->getId() == v->getId()) {
                return true;
            }
        }
        return false;
    }
```

The `mergeGraphs` method starts off in a familiar fashion. Find the largest size between the first and second graphs, and create a new `Graph` object with this size. Then iterate over the first graph, adding each edge to the merged graph.

```
    static Graph mergeGraphs(const Graph& graph1, const Graph&
        graph2) {
        int totalVertices = std::max(graph1.getSize(),
        graph2.getSize());
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); ++i) {
            Vertex* vertex = graph1.getVertices()[i];
            std::vector<Vertex*> neighbors = vertex-
        >getNeighbors();
            for (Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
            }
        }



    }
```

Things start to differ when merging the second graph. Start this process as before. Iterate over the second graph and generate a list of neighbors for the current vertex (second graph) and a list of neighbors for the same vertex in the merged graph. Iterate over the list of neighbors (second graph) and see if the current neighbor is in the list of neighbors in the merged graph. If not, add the edge. When the loops are done, return the merged graph.

```
        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); ++i) {
            Vertex* vertex = graph2.getVertices()[i];
            std::vector<Vertex*> neighbors = vertex-
        >getNeighbors();
            std::vector<Vertex*> mergedNeighbors =
        mergedGraph.getVertices()[i]->getNeighbors();
            for (Vertex* neighbor : neighbors) {
                if (containsEdge(neighbor, mergedNeighbors)) {



                } else {
                    addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
                }
            }
        }

        return mergedGraph;
    }
```

If the current neighbor is in the list of neighbors for the merged graph, then we have a duplicate edge. Get the corresponding vertex from the merged graph. Then get the index for the neighbor (second graph). Get the

existing weight for the vertex in the merged graph and the weight of the current vertex in the second graph. Add these weights together. Finally, update the weight for the vertex in the merged graph.

```
    if (containsEdge(neighbor, mergedNeighbors)) {
        Vertex* mergedVertex = mergedGraph.getVertices()[i];
        auto it = std::find(neighbors.begin(),
      neighbors.end(), neighbor);
        int neighborIndex = std::distance(neighbors.begin(),
      it);
        int existingWeight = mergedVertex-
      >getWeight(neighborIndex);
        int currentWeight = vertex-
      >getWeight(std::distance(neighbors.begin(),
      std::find(neighbors.begin(), neighbors.end(),
      neighbor)));
        int newWeight = existingWeight + currentWeight;
        mergedVertex->updateWeight(neighborIndex, neighbor,
      newWeight);
    } else {
        addEdge(mergedGraph, i, vertex, neighbor, neighbors);
    }
```

▼ **Code**

> Your code should look like this:
>
> ```
> class GraphMerger {
>
> public:
>     static void addEdge(Graph& graph,
>         int index,
>         Vertex* currentVertex,
>         Vertex* neighbor,
>         const std::vector<Vertex*>& neighbors
>     ) {
>         int sourceIndex = index;
>         int targetIndex = neighbor->getId();
>         auto it = std::find(neighbors.begin(),
>         neighbors.end(), neighbor);
>         int neighborIndex =
>         std::distance(neighbors.begin(), it);
>         int weight = currentVertex-
>         >getWeight(neighborIndex);
>         graph.addEdge(sourceIndex, targetIndex, weight);
>     }
>
>     static bool containsEdge(Vertex* v, const
>         std::vector<Vertex*>& mergedEdges) {
>         for (const auto& edge : mergedEdges) {
>             if (edge->getId() == v->getId()) {
>                 return true;
>             }
>         }
>         return false;
>     }
> ```

```
    static Graph mergeGraphs(const Graph& graph1, const
        Graph& graph2) {
        int totalVertices = std::max(graph1.getSize(),
        graph2.getSize());
        Graph mergedGraph(totalVertices);

        // Add vertices and edges from graph1
        for (int i = 0; i < graph1.getSize(); ++i) {
            Vertex* vertex = graph1.getVertices()[i];
            std::vector<Vertex*> neighbors = vertex-
        >getNeighbors();
            for (Vertex* neighbor : neighbors) {
                addEdge(mergedGraph, i, vertex, neighbor,
        neighbors);
            }
        }

        // Add vertices and edges from graph2
        for (int i = 0; i < graph2.getSize(); ++i) {
            Vertex* vertex = graph2.getVertices()[i];
            std::vector<Vertex*> neighbors = vertex-
        >getNeighbors();
            std::vector<Vertex*> mergedNeighbors =
        mergedGraph.getVertices()[i]->getNeighbors();
            for (Vertex* neighbor : neighbors) {
                if (containsEdge(neighbor,
        mergedNeighbors)) {
                    Vertex* mergedVertex =
        mergedGraph.getVertices()[i];
                    auto it = std::find(neighbors.begin(),
        neighbors.end(), neighbor);
                    int neighborIndex =
        std::distance(neighbors.begin(), it);
                    int existingWeight = mergedVertex-
        >getWeight(neighborIndex);
                    int currentWeight = vertex-
        >getWeight(std::distance(neighbors.begin(),
        std::find(neighbors.begin(), neighbors.end(),
        neighbor)));
                    int newWeight = existingWeight +
        currentWeight;
                    mergedVertex-
        >updateWeight(neighborIndex, neighbor, newWeight);
                } else {
                    addEdge(mergedGraph, i, vertex,
        neighbor, neighbors);
                }
            }
        }

        return mergedGraph;
    }
};
```

Run the program. The merged graph should have neighbors for vertices `0`, 1, 2, 4, and 5. However, since vertices 1 and 2 are overlapping, their weights should be updated.

You should see the following output:

```
Graph 1:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (3)
Vertex 2: -> 4 (5)
Vertex 3:
Vertex 4:
Vertex 5:

Graph 2:
Vertex 0:
Vertex 1: -> 2 (4)
Vertex 2: -> 4 (1)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)

Merged Graph:
Vertex 0: -> 1 (2) -> 3 (1)
Vertex 1: -> 2 (7)
Vertex 2: -> 4 (6)
Vertex 3:
Vertex 4: -> 5 (0)
Vertex 5: -> 3 (3)
```

challenge

Try this variation

Update the `mergeGraphs` method so that you choose the largest weight instead of combining the weights.

▼ **Solution**

> Update the line of code that adds together `existingWeight` and `currentWeight` so that it uses `std::max` to select the largest weight:
>
> ```
> int newWeight = std::max(existingWeight,
>         currentWeight);
> ```

Handling overlapping vertices in graph merging is a nuanced task that requires careful consideration of the graph's structure and the data associated with their vertices. The chosen strategy should align with the overall goal of the merging process and the intended use of the merged graph

Formative Assessment 1

Formative Assessment 2